

神志病AI问诊训练工作台 正式网页版部署操作指南

这份指南用于把当前本地项目部署成可分享访问的网址：前端部署到 Vercel/Netlify，后端部署到 Render/Railway/Fly.io/云服务器，并完成 DeepSeek API Key、环境变量、健康检查和上线验证。

推荐路线

GitHub + Render 后端 +
Vercel 前端

当前项目路径

/Users/qymeded/Desktop/shenzhi_agent_mvp

核心后端文件

api_server.py

核心前端目录

frontend/

版本日期 2026-06-06

适用对象 项目维护者、课堂/比赛演示负责人、后续协作者

交付目标 别人打开一个网址即可使用网页端问诊训练系统

目录

1. 你最终会得到什么
2. 部署架构说明
3. 上线前准备清单
4. 本地运行确认
5. 上传项目到 GitHub
6. 部署后端到 Render
7. 部署前端到 Vercel
8. Netlify 前端备选方案
9. 上线后的完整验证流程
10. 后续更新怎么发布
11. 常见问题排查
12. 公开分享前的安全提醒

1. 你最终会得到什么

完成部署后，你会有两个线上地址：

地址类型	示例	用途
前端网址	https://shenzhi-agent.vercel.app	发给别人访问。用户看到的网页 UI、聊天界面、评分面板都在这里。
后端网址	https://shenzhi-agent-api.onrender.com	前端背后调用的 API 服务。负责病例、问诊、评分、PDF 导出、DeepSeek 调用。

一句话理解：前端是“网页外壳和操作界面”，后端是“系统大脑和数据接口”。别人只需要打开前端网址，前端会自动请求后端。

2. 部署架构说明



项目中已经做过部署适配：

- 前端通过 `VITE_API_BASE_URL` 指向线上后端。
- 后端通过平台提供的 `PORT` 自动监听正确端口。
- PDF 导出、舌象图片等资源链接也会跟随后端域名。

3. 上线前准备清单

3.1 账号准备

- ☐ GitHub 账号：用于保存代码仓库。
- ☐ Render 账号：用于部署 Python 后端。
- ☐ Vercel 账号：用于部署 React/Vite 前端。
- ☐ DeepSeek API Key：用于后端调用模型。

3.2 本机工具准备

工具	用途	检查命令
Git	提交并上传代码到 GitHub	<code>git --version</code>
Python	本地运行后端和测试	<code>python3 --version</code>
Node.js / pnpm	本地运行和构建前端	<code>node -v</code> 、 <code>pnpm -v</code>

如果提示 `pnpm: command not found`，建议先安装 Node.js LTS，然后执行：

```
corepack enable
corepack prepare pnpm@latest --activate
pnpm -v
```

3.3 不要提交到 GitHub 的内容

这些内容不能上传，尤其是 API Key：

- `.venv/`：本地 Python 虚拟环境。
- `frontend/node_modules/`：本地前端依赖。
- `frontend/dist/`：构建产物，平台会自动生成。
- `.streamlit/secrets.toml`：本地密钥文件。
- `runtime/`：本地运行时数据库。
- `reports/`：本地导出的报告。

项目中的 `.gitignore` 已经排除了这些内容，但上传前仍建议用 `git status` 看一眼。

4. 本地运行确认

正式部署前，先确认本地版本能正常运行。打开终端：

```
cd /Users/qymeded/Desktop/shenzhi_agent_mvp
```

4.1 启动后端

```
source .venv/bin/activate
./scripts/start_api.sh
```

后端默认地址：

```
http://127.0.0.1:8765
```

另开一个终端检查健康接口：

```
curl http://127.0.0.1:8765/api/health
```

正常返回：

```
{"status": "ok"}
```

4.2 启动前端

```
cd /Users/qymeded/Desktop/shenzhi_agent_mvp/frontend
pnpm install
pnpm dev
```

前端默认地址：

```
http://localhost:5173
```

打开后做三个动作：新建问诊、发送一句问题、导出问诊记录 PDF。都正常再进入部署。

5. 上传项目到 GitHub

5.1 如果你还没有 GitHub 仓库

1 在 GitHub 新建仓库

进入 GitHub，点击 New repository。仓库名可以叫 `shenzhi_agent_mvp` 或 `shenzhi-ai-workbench`。不要勾选自动创建 README，因为本地已经有项目文件。

2 在本地初始化并提交

```
cd /Users/qymeded/Desktop/shenzhi_agent_mvp
git init
git add .
git commit -m "Prepare formal web deployment"
git branch -M main
```

3 关联远程仓库并推送

```
git remote add origin https://github.com/你的用户名/你的仓库名.git
git push -u origin main
```

5.2 如果你已经有 GitHub 仓库

```
cd /Users/qymeded/Desktop/shenzhi_agent_mvp
git status
git add .
git commit -m "Prepare formal web deployment"
git push
```

注意：如果 `git status` 里出现 `.streamlit/secrets.toml`、`.venv`、`node_modules`，先不要提交，说明忽略规则可能没生效，需要先处理。

6. 部署后端到 Render

后端负责所有核心功能：病例读取、问诊对话、评分、PDF 导出、模型调用。因此后端必须先部署成功，前端才能正确连接。

6.1 创建 Web Service

1 登录 Render

进入 Render 控制台，点击 **New +**，选择 **Web Service**。

2 连接 GitHub 仓库

选择你刚刚上传的项目仓库。如果第一次使用，需要授权 Render 读取 GitHub。

3 填写服务配置

配置项	填写内容
Name	shenzhi-agent-api
Root Directory	留空，表示项目根目录
Runtime	Python
Branch	main
Build Command	pip install -r requirements.txt
Start Command	python api_server.py
Health Check Path	/api/health

6.2 配置环境变量

在 Render 的 Environment 页面添加：

Key	Value
DEEPSEEK_API_KEY	你的 DeepSeek API Key。不要写进代码，不要上传 GitHub。
SHENZHI_RUNTIME_DIR	/opt/render/project/src/runtime

Render 会自动提供 `PORT` 环境变量。当前后端代码已经支持它：线上会监听 `0.0.0.0:$PORT`，本地仍默认监听 `127.0.0.1:8765`。

6.3 部署并检查

1. 点击 Deploy Web Service。
2. 等待日志里出现类似 `Shenzhi API listening`。
3. 打开后端健康检查地址：

<https://你的后端域名.onrender.com/api/health>

正常返回：

```
{"status": "ok"}
```

6.4 关于持久化数据

当前问诊历史会写到运行目录中的 SQLite 数据库。Render 如果没有挂持久化磁盘，服务重启或重新部署后，运行时数据可能丢失。

演示阶段可以先不挂磁盘；如果要长期给别人用，建议在 Render 给服务添加 Persistent Disk，然后把 `SHENZHI_RUNTIME_DIR` 设置为磁盘挂载目录。

7. 部署前端到 Vercel

前端是用户真正访问的网址。它不保存 DeepSeek API Key，只通过环境变量知道后端在哪。

7.1 导入项目

1 登录 Vercel

进入 Vercel 控制台，点击 **Add New Project** 或 **Import Project**。

2 选择 GitHub 仓库

选择同一个 `shenzhi_agent_mvp` 仓库。

3 填写构建配置

配置项	填写内容
Framework Preset	Vite
Root Directory	<code>frontend</code>
Build Command	<code>pnpm build</code>
Output Directory	<code>dist</code>

7.2 设置前端环境变量

在 Vercel 的 Environment Variables 里添加：

```
VITE_API_BASE_URL=https://你的后端域名.onrender.com
```

注意：

- 不要在末尾加 `/`，例如不要写成 `https://xxx.onrender.com/`。
- 改了这个环境变量后，必须重新部署前端，因为 Vite 会在构建时写入变量。

7.3 发布并访问

- 点击 Deploy。
- 构建成功后，Vercel 会给你一个前端网址。
- 打开前端网址，确认能看到登录/工作台界面。

如果页面能打开但问诊失败：多数是 `VITE_API_BASE_URL` 配错、后端没启动、后端 API Key 没配，或者前端改环境变量后没有重新部署。

8. Netlify 前端备选方案

如果你不用 Vercel，也可以把前端部署到 Netlify。配置如下：

配置项	填写内容
Base directory	<code>frontend</code>
Build command	<code>pnpm build</code>
Publish directory	<code>dist</code>
Environment variable	<code>VITE_API_BASE_URL=https://你的后端域名.onrender.com</code>

前端平台二选一即可。推荐先用 Vercel，因为 Vite 项目的默认识别和部署体验通常更顺。

9. 上线后的完整验证流程

部署完成后不要只看首页，要按下面清单逐项验证。

9.1 后端验证

- ☐ 打开 `/api/health`，返回 `{"status":"ok"}`。
- ☐ Render 日志没有持续报错。
- ☐ 环境变量里已经填入 `DEEPSEEK_API_KEY`。

9.2 前端验证

- ☐ 前端网址能打开。
- ☐ 登录界面/工作台 UI 显示正常。
- ☐ 能进入问诊页面。
- ☐ 左侧历史记录、中间对话、右侧评分面板显示正常。

9.3 功能验证

功能	验证动作	通过标准
新建问诊	点击新建会话或进入默认会话。	能看到病例标题和聊天输入框。
患者回复	发送“你最近主要哪里不舒服？”	患者 Agent 返回符合病例的自然回答。
评分反馈	连续问 2-3 个问题。	本轮评分和右侧评分详情能更新。
督导反馈	在右侧督导区域提问。	能给出下一步问诊建议。
PDF 导出	点击导出问诊记录。	下载 PDF，内容包含本轮全部问诊记录。

9.4 Network 检查

打开浏览器开发者工具，进入 Network。刷新页面后查看请求：

- `/api/chats` 应该请求到 Render 后端域名。
- 如果请求跑到 Vercel/Netlify 自己的域名，说明 `VITE_API_BASE_URL` 没生效。
- 如果接口返回 500，去 Render Logs 看后端错误。

10. 后续更新怎么发布

后续修改 UI、评分逻辑、患者 Agent 提示词或 PDF 导出后，按下面流程发布。

```
cd /Users/qymeded/Desktop/shenzhi_agent_mvp
git status
git add .
```

```
git commit -m "Describe your update"
git push
```

推送后：

- Vercel 通常会自动重新部署前端。
- Render 如果开启 Auto Deploy，也会自动重新部署后端。
- 如果只改了后端，前端不一定需要重新部署。
- 如果改了 `VITE_API_BASE_URL`，必须重新部署前端。

建议发布节奏：每次只改一个主题，例如“登录 UI 优化”“评分逻辑优化”“PDF 导出优化”。这样线上出问题时容易回退。

11. 常见问题排查

问题 1：前端页面打开了，但发送消息失败。

先检查后端健康接口是否正常，再检查 Vercel 的 `VITE_API_BASE_URL` 是否是 Render 后端域名。改完环境变量后要重新部署前端。

问题 2：Render 后端部署失败。

看 Render Logs。重点检查 `Build Command` 是否是 `pip install -r requirements.txt`，`Start Command` 是否是 `python api_server.py`。

问题 3：后端显示启动了，但健康检查打不开。

通常是服务没有监听平台端口。当前代码已经支持 `PORT`，如果仍失败，确认线上运行的是最新代码，并检查日志里的监听地址。

问题 4：AI 不回复或一直报错。

检查 `DEEPSEEK_API_KEY` 是否填在 Render 后端环境变量中，而不是 Vercel 前端。还要检查 DeepSeek 账号额度和服务状态。

问题 5：PDF 导出失败。

确认后端服务正常，当前会话确实有问诊记录。Network 里应看到类似 `/api/chats/xxx/export` 的请求，响应类型应该是 `application/pdf`。

问题 6：历史记录重启后消失。

这是没有配置持久化磁盘导致的。演示阶段可以接受；长期使用需要 Render Persistent Disk 或外部数据库。

问题 7：Vercel 构建失败，提示 pnpm 找不到。

在 Vercel 构建配置里保持 `pnpm build`。如果平台没有自动识别，可改成 `corepack enable && pnpm install && pnpm build`，或在项目中补充平台配置。

问题 8：浏览器提示 CORS。

当前后端默认允许跨域。如果后续改了 CORS 限制，需要把你的 Vercel/Netlify 前端域名加入允许列表。

12. 公开分享前的安全提醒

重要：当前登录界面主要是 UI 状态，不是真正的后端账号鉴权。只要别人知道前端网址，就可能使用后端接口并消耗你的 DeepSeek API Key。

正式公开分享前，建议至少做一种保护：

- 给后端加真实登录鉴权。
- 给演示版本加访问密码。
- 限制 CORS，只允许指定前端域名访问。
- 设置请求频率限制，避免多人同时大量调用。
- 演示阶段只把链接发给小范围测试用户。

推荐的正式上线顺序

1. 先部署后端，并确认 `/api/health` 正常。
2. 再部署前端，并设置 `VITE_API_BASE_URL`。
3. 自己完整走一遍问诊和 PDF 导出。
4. 发给 1-2 个熟人测试。
5. 确认稳定后再扩大分享范围。

本指南配套当前项目版本生成。若后续改成真实账号系统、数据库服务或云服务器部署方式，应同步更新本指南。